

Omnibit Index Forge

Base Sepolia MVP Technical Specification — v1.2

Field	Value
Document version	1.2 (supersedes v1.0 and v1.1 where they conflict)
Product	Omnibit Index Forge — create fully backed, accretive index tokens on Base
Reference index	Index AI2 — 50% tNVDA / 50% tMSFT (synthetic B20 test assets)
Target network	Base Sepolia only — chain ID 84532
Status	Binding MVP implementation specification
Requirement language	MUST / MUST NOT mandatory. SHOULD default unless documented. MAY optional.
Audience	Implementing Solidity engineer and coding agents. Not a pitch deck.
Testnet disclaimer	tNVDA and tMSFT have no real-world backing or value.

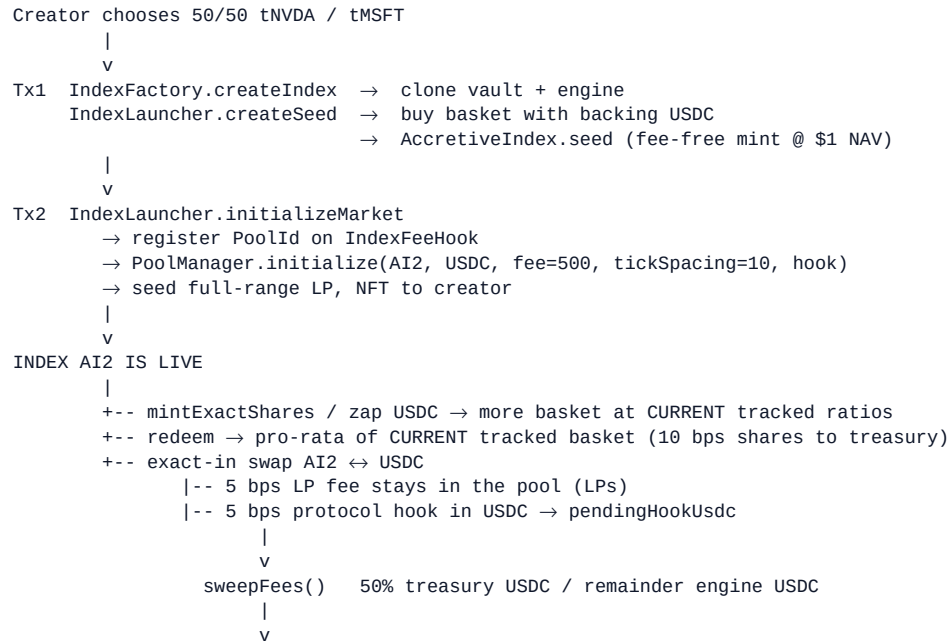
Read this page first

You are building a vault + factory + Uniswap V4 hook that issues a fully backed basket token (Index AI2) and uses swap fees to buy more of the same basket without minting new shares. Reuse audited libraries and live Base/Uniswap infrastructure. Write the protocol-specific contracts listed in §0.3. Do not invent a third design. If a behavior is not written here, it is out of scope.

0. Solidity briefing — what you are building

Omnibit Index Forge is a permissionless index-token launch protocol. A creator picks approved B20 constituents and launch weights. The protocol clones a vault (AccretiveIndex) and an accretion engine, buys the basket with USDC, mints fully backed Index AI2 shares at ~\$1 NAV, and initializes a canonical AI2/USDC Uniswap V4 pool with a global protocol hook. After launch, anyone can mint in-kind, redeem in-kind, or trade on that pool. Hook fees in USDC are swept 50/50 to treasury and the engine. The engine harvests by buying more tNVDA and tMSFT at the original 50/50 launch weights and depositing them with zero new shares. Redeemers later take a larger pile of the same assets.

0.1 System workflow (one picture)



```

harvest()      spend engine USDC 50/50 at LAUNCH weights
  |
  v
depositAccretion()  worst-leg recognized, ZERO shares minted
  |
  v
backing per share up; totalSupply unchanged

```

0.2 v1.2 decisions that override v1.0 / v1.1

Topic	v1.2 rule
Basket	Index AI2 is 50/50 tNVA / tMSFT. Vault accounting remains N-generic (MAX_CONSTITUENTS = 8).
Harvest spend	Engine spends USDC at launch weights (50/50), same as the launch buy — not live value-weights.
Worst-leg	Still applies. If one leg cannot fully fill, only the bottleneck amount is recognized. Leftover stays in the engine.
LP fee vs hook	5 bps V4 LP fee stays with LPs. 5 bps protocol hook is USDC-only and is the only fee that is swept.
Mint / redeem fee	10 bps in Index AI2 shares to treasury. Never USDC. Never harvested.
Launch	Two transactions: createSeed, then initializeMarket. Hook register + PoolManager.initialize stay launcher-owned.
Reference engine	TypeScript simulator (engine.ts + engine.test.ts) is the golden-vector source for mint/redeem/harvest math.

0.3 Reuse vs build-from-scratch (do this first)

Copying the wrong thing, or rewriting audited libraries, is how this slips. Keep original copyright headers on any adapted MIT file.

Component	Source	What you do
ERC-20, SafeERC20, Math.mulDiv, ReentrancyGuard, Clones, Initializable, AccessControl	OpenZeppelin Contracts / Upgradeable	Depend. Do not rewrite.
IB20, IB20Asset, IB20Factory, B20FactoryLib, B20Constants, StdPrecompiles	base/base-std (pin commit)	Depend. Pin the commit in foundry.toml / remappings.
tNVA / tMSFT test tokens	Live Base Sepolia B20 Factory 0xB20f...0000	createB20 ASSET via the precompile. Do not deploy homemade ERC-20s named tNVA.
USDC (6 dec)	Circle test USDC 0x036CbD53842c5426634e7929541eC2318f3dCF7e	Use the existing token. Do not deploy a fake USDC.
PoolManager, PositionManager, Universal Router, Quoter, StateView, TickMath, PoolKey	Official Uniswap V4 on Base Sepolia	Use deployed addresses + v4-core / v4-periphery libraries. Do not fork an AMM.
Permit2	0x000000000022D473030F116dDEE9F6B43aC78BA3	Use as-is for launch approvals.
Hook base / settlement helpers	OpenZeppelin Uniswap Hooks	Depend where useful. Fee policy and registry are ours.
Factory / router / test patterns	Vimen (MIT) https://github.com/vimenprotocol/vimen	Adapt patterns and tests if helpful. Keep license headers.
Pro-rata vault idea	Common vault pattern (HoodETF-class references)	Independent implementation. Do not paste unaudited vault guts.
AccretiveIndex.sol	This protocol	BUILD. Share token + custody + mint/redeem/accretion/syncLoss.
IndexFactory.sol	This protocol	BUILD on OZ Clones. Asset registry. onlyLauncher createIndex.
IndexLauncher.sol	This protocol	BUILD. Two-tx createSeed + initializeMarket.
IndexZapRouter.sol	This protocol	BUILD. USDC convenience + buyTargetBasket.

Component	Source	What you do
UniswapV4SwapAdapter.sol	This protocol	BUILD. Allowlisted exact-in constituent/USDC swaps only.
IndexFeeHook.sol	This protocol	BUILD. CREATE2-mined global hook. USDC custom-accounting fee + registry + sweep.
AccretionEngine.sol	This protocol	BUILD. Sweep USDC in → 50/50 buys → depositAccretion (zero shares).
NavLib.sol / SqrtPriceLib.sol	This protocol	BUILD. WAD NAV and initialize sqrtPriceX96.
MockPriceFeed.sol	This protocol	BUILD. Sepolia-only AggregatorV3 mocks.
Frontend / TS engine	This protocol (already prototyped)	Reference math lives in engine.ts. On-chain must match those vectors.

Contents

- 0. Solidity briefing, workflow, reuse matrix (this page)
- 1. Scope, success criteria, economic loop
- 2. Fees and flow of funds
- 3. Base Batches 004 alignment
- 4. Architecture and verified addresses
- 5. Test assets, units, NAV
- 6. AccretiveIndex, Factory, Launcher
- 7. Zap, adapter, hook, engine
- 8. Invariants, tests, deployment
- 9. Frontend and acceptance
- Appendix A. Reference URLs
- Appendix B. Worked 50/50 numeric sketch
- Appendix C. Agent implementation notes

1. Scope, success criteria, and economic loop

Build a permissionless index-token launch protocol on Base Sepolia. A creator selects approved B20 constituents and initial weights. The protocol creates a fully backed ERC-20 index share, acquires the underlying B20 assets, initializes a canonical INDEX/USDC Uniswap V4 pool with the global protocol hook attached, and seeds initial liquidity.

The MVP reference index is Index AI2 with two constituents at 50/50: tNVDA and tMSFT. Core contracts MUST be N-asset generic so a later MAG7 configuration requires new parameters and liquidity, not a rewrite. MAX_CONSTITUENTS = 8.

1.1 Definition of MVP success

- Real B20 test assets are created through Base's live B20 Factory on Base Sepolia.
- A factory-created Index AI2 token is fully backed by tNVDA and tMSFT.
- In-kind mint and redeem work without an oracle or DEX dependency.
- Two-tx launch initializes a canonical AI2/USDC V4 pool using the global protocol hook and seeds liquidity.
- Canonical exact-input swaps generate a 5 bps USDC protocol hook fee on both buy and sell, separate from the 5 bps LP fee.
- sweepFees splits pending hook USDC 50% treasury / remainder AccretionEngine.
- harvest spends engine USDC 50/50 at launch weights, then depositAccretion recognizes only worst-leg amounts and mints zero shares.
- Accretion increases backing per Index AI2 without increasing totalSupply.
- A holder redeeming after accretion receives their pro-rata share of the larger basket.
- Unit, fuzz, invariant, and Base Sepolia integration tests in §8 pass.
- Frontend shows pre- and post-accretion backing-per-share using on-chain cumulativeAccretedUsdWad.

1.2 What “done” is not

Done is not a mainnet launch, not MAG7, not a keeper network, and not a governance token. Done is a replayable Sepolia demonstration of issuance, canonical V4 trading, fee split, harvest, and post-accretion redeem.

2. Fees and flow of funds

Action	Protocol / pool fee	Paid in	Where it goes
Buy/sell Index AI2 in the canonical AI2/USDC pool	5 bps protocol hook	USDC	Pending claim. sweepFees: 50% treasury, remainder accretion engine.
Same swap, LP cut	5 bps V4 LP fee (fee=500)	Pool units	Stays in the pool for LP providers. NEVER swept. NEVER harvested.
Mint Index AI2	10 bps	Index AI2 shares	100% protocolTreasury. Fully backed. Not USDC.
Redeem Index AI2	10 bps	Index AI2 shares	100% protocolTreasury. Fully backed. Not USDC.
seed()	0	—	Launcher-only, fee-free.

Integer split on sweep: $\text{treasuryCut} = \text{pending} / 2$; $\text{engineCut} = \text{pending} - \text{treasuryCut}$. Odd wei goes to the engine. Harvest USDC is only the engine’s half of hook fees after a sweep. Mint and redeem never feed harvest.

swaps $\rightarrow$ 5 bps USDC hook $\rightarrow$ sweep $\rightarrow$ engine USDC $\rightarrow$ harvest $\rightarrow$ more tNVDA + tMSFT in vault
 ■ treasury keeps half the USDC
 5 bps LP fee remains in the pool
 mint / redeem $\rightarrow$ 10 bps shares $\rightarrow$ treasury

3. Base Batches 004 alignment

Program filter	How this MVP maps
Base-first	Every contract, asset, pool, and script targets Base Sepolia 84532.
Asset issuance + tokenized equities	Live B20 Factory ASSET variant as constituents.
RFB: Personalized Index Creation	Index AI2 is the two-asset proof. Factory is N-generic.
Trading	Canonical INDEX/USDC Uniswap V4 pool with a protocol hook.
Demo Day narrative	Launch Index AI2 from USDC $\rightarrow$ trade $\rightarrow$ sweep $\rightarrow$ harvest $\rightarrow$ redeem more basket per share.

Grant-risk warning

Do not imply tNVDA/tMSFT are Coinbase-issued live tokenized stocks. They are synthetic B20 test assets. Live Coinbase B20 stocks are a later constituent-approval exercise. The MVP proves the primitive on test B20s.

4. Architecture and verified addresses

Layer	Component	Responsibility
Asset	B20 tNVDA / tMSFT	Synthetic test constituents from the live Base B20 Factory.
Core	AccretiveIndex	ERC-20 share + custody vault. Tracked balances, mint/redeem, accretion.
Factory	IndexFactory	Approved-asset registry. Clones one index + one engine. onlyLauncher.
Launch	IndexLauncher	Two-step createSeed + initializeMarket.
Convenience	IndexZapRouter	USDC $\rightarrow$ basket $\rightarrow$ mint; launch buyTargetBasket.
Swap	UniswapV4SwapAdapter	Allowlisted exact-in constituent/USDC swaps only.
Market	Uniswap V4 PoolManager	Canonical INDEX/USDC state. No per-index pool contract.
Fee	IndexFeeHook	One global hook. USDC protocol fee. Pool registry. 50/50 sweep.

Layer	Component	Responsibility
Accretion	AccretionEngine	One per index. USDC in → 50/50 constituents → depositAccretion.
Valuation	MockPriceFeed	Chainlink-compatible mocks. Not used for redeem claims.

A V4 market is initialized inside the singleton PoolManager. IndexLauncher does not deploy a new pool contract. It constructs a PoolKey (INDEX, USDC, fee 500, tickSpacing 10, global IndexFeeHook). PoolId = keccak of that PoolKey.

4.1 Verified Base Sepolia addresses — re-check before broadcast

Dependency	Base Sepolia address
Chain	84532
B20 Factory	0xB20f00
Circle test USDC (6 dec)	0x036CbD53842c5426634e7929541eC2318f3dCF7e
Uniswap V4 PoolManager	0x05E73354cFDd6745C338b50BcFDfA3Aa6fA03408
Uniswap V4 Universal Router	0x492e6456d9528771018deb9e87ef7750ef184104
Uniswap V4 PositionManager	0x4b2c77d209d3405f41a037ec6c77f7f5b8e2ca80
Uniswap V4 StateView	0x571291b572ed32ce6751a2cb2486ebee8defb9b4
Uniswap V4 Quoter	0x4a6513c898fe1b2d0e78d3b0e0a4a151589b1cba
Permit2	0x000000000022D473030F116dDEE9F6B43aC78BA3

5. Test assets, units, NAV

Asset	Type	Decimals	Initial mock USD
tNVDA	B20 ASSET	18	\$200.00 (feed answer 200_00000000)
tMSFT	B20 ASSET	18	\$500.00 (feed answer 500_00000000)
USDC	Circle test	6	\$1.00 by convention, no feed
Index AI2	protocol ERC-20	18	NAV-derived

- Create tNVDA and tMSFT through the live B20 Factory, variant ASSET, labeled test-only. Fixed published salts.
- Core ownership uses raw ERC-20 balanceOf(). B20 multiplier() is only for NAV / display.
- Happy-path tokens MUST have no transfer policies, MUST be unpaused, MUST NOT be fee-on-transfer.
- Feeds MUST NOT determine redeem rights. Stale feeds MAY block zap and harvest; they MUST NOT block in-kind redeem.
- Pre-launch, seed reusable V4 test markets (tNVDA/USDC and tMSFT/USDC). Record those PoolKeys.
- Normal Foundry anvil does not host B20 precompiles. Unit tests use mocks. Integration uses a Base Sepolia fork or base-anvil.

5.1 Units

Unit	Scale	Rule
INDEX / Index AI2	18	MUST be 18. Do not match USDC.
USDC	6	All USDC function args are 6-dec raw.
Constituents	18	Raw vault units.
Feed answer	8	MockPriceFeed.decimals() = 8.
NAV, navPerShare, scaleFactor	WAD 1e18	Integer only. No floating point.
Fees	bps / 10_000	5 hook, 5 LP, 10 mint, 10 redeem.

NAV_WAD = sum_i usdValueWad(economicAmount(tracked[i]), 18, feedAnswer[i], 8)

```

navPerShareWAD = NAV_WAD * 1e18 / totalSupply          // totalSupply == 0 → revert
// sqrtPriceX96: currencies are address-sorted. INDEX may be token0 or token1.
// Encode price of currency0 in units of currency1 using each token's native decimals.

```

6. AccretiveIndex, Factory, Launcher

6.1 AccretiveIndex.sol — BUILD

Each clone is the ERC-20 share token and the custody address for recognized constituents. Implementation constructor **MUST** call `_disableInitializers()`.

```

address[] public constituents;
mapping(address => uint256) public trackedBalance;
mapping(address => uint256) public cumulativeAccretedRaw;
uint256 public cumulativeAccretedUsdWad;
address public factory; address public launcher;
address public accretionEngine; address public protocolTreasury;
uint16 public mintFeeBps;          // 10
uint16 public redeemFeeBps;       // 10
uint8  public constant DECIMALS = 18;
bool public seeded; bool public mintPaused; bool public closed;

// Invariant: one share owns tracked[i] / totalSupply.
// Only seed, mintExactShares, redeem, depositAccretion, syncLoss may change tracked.
// A raw ERC-20 transfer into the vault is not backing.

function seed(uint256[] calldata amounts, uint256 initialGrossShares, address lpReceiver) external;
function mintExactShares(uint256 grossShares, address receiver)
    external returns (uint256 userShares, uint256 feeShares);
function redeem(uint256 sharesIn, address receiver) external returns (uint256[] memory assetOut);
function depositAccretion(uint256[] calldata amounts) external; // engine only, ZERO mint
function syncLoss(address asset) external; // permissionless, never increases tracked

// mint
requiredAsset[i] = mulDiv(tracked[i], grossShares, totalSupplyBefore, Ceil);
feeShares        = grossShares * mintFeeBps / 10_000;
userShares       = grossShares - feeShares;
_mint(receiver, userShares);
_mint(protocolTreasury, feeShares);

// redeem
feeShares        = sharesIn * redeemFeeBps / 10_000;
redeemShares     = sharesIn - feeShares;
assetOut[i]      = mulDiv(tracked[i], redeemShares, totalSupplyBefore, Floor);
// transfer feeShares INDEX to treasury; burn redeemShares; send assetOut

```

- seeded is set once by seed(), onlyLauncher. False → mint / redeem / depositAccretion revert.
- mintPaused: mint + zap mint revert. Redeem, sweep, harvest, syncLoss still work.
- depositAccretion **MUST NOT** call `_mint`, directly or indirectly.
- syncLoss pulls tracked down to raw balanceOf. Required for the B20 seize case.

6.2 IndexFactory.sol — BUILD on OZ Clones

```

struct CreateIndexParams {
    string  name;          // "Index AI2"
    string  symbol;        // "AI2"
    address[] constituents;
    uint16[] initialWeightsBps; // MUST sum to 10_000; AI2 = [5000, 5000]
    address creator;
}
function createIndex(CreateIndexParams calldata p)
    external returns (address index, address accretionEngine); // onlyLauncher

```

- N in [2, 8], equal-length arrays, no duplicates, every constituent approved.
- **SHOULD** verify constituents are B20 ASSETs known to the B20 Factory.
- Factory initializes both clones and registers the new engine as an allowed adapter caller.
- approveAsset / setPriceFeed / revokeAsset are guardian-only. Reject fee-on-transfer assets.

6.3 IndexLauncher.sol — BUILD, two transactions

```
function createSeed(
    CreateIndexParams calldata indexParams,
    uint256 backingUSDC, uint256 maxBackingUSDC, uint256 deadline
) external returns (address index, address accretionEngine, uint256 grossShares);
```

```
function initializeMarket(
    address index, uint256 lpUSDC, uint256 maxLpUSDC, uint256 deadline
) external returns (PoolId poolId, uint256 positionTokenId);
```

- createSeed: factory.createIndex → pull backingUSDC → buyTargetBasket at launch weights → seed fee-free so 1 AI2 ≈ \$1 → refund unused USDC.
- initializeMarket: require seeded; pull lpUSDC; register PoolId on hook; PoolManager.initialize; full-range PositionManager mint; NFT to creator.
- Launch weights are the buy target for seed and later harvest. After slippage, live tracked balances are whatever actually arrived. Later mints follow current tracked ratios.
- Hook registration and PoolManager.initialize MUST stay launcher-owned.

7. Zap, adapter, hook, engine

7.1 IndexZapRouter + UniswapV4SwapAdapter — BUILD

- mintExactSharesWithUSDC(index, desiredGrossShares, maxUSDC, deadline).
- buyTargetBasket(constituents, weightsBps, usdcIn, minOut[], deadline) — used by launcher and harvest.
- redeemToUSDC is secondary. Solvency MUST remain available by calling AccretiveIndex directly with constituent tokens.
- Adapter: swapExactInput only. Only router + registered engines. Only registered constituent/USDC PoolKeys.
- One PoolManager.unlock callback inside the adapter. Guard unlockCallback. No arbitrary calldata.

7.2 IndexFeeHook.sol — BUILD, highest risk

One global hook for every canonical index pool. Deploy with CREATE2 / HookMiner so permission bits are in the address. Implement against a local v4-core PoolManager harness first. Do not iterate only on Sepolia.

```
beforeInitialize, beforeSwap, afterSwap,
beforeSwapReturnDelta, afterSwapReturnDelta = true
all other flags = false
constructor args: PoolManager only. Mine salt, then freeze bytecode.
```

```
Fee algorithm (exact-in only; reject exact-out on registered pools):
  buy (USDC → AI2): take 5 bps of USDC input before the swap
  sell (AI2 → USDC): take 5 bps of USDC output after the swap
Store pendingHookUsdc[poolId]. Never take AI2 as the protocol fee.
```

```
sweepFees(poolId):
  treasury = pending / 2
  engine = pending - treasury
  pending = 0
  pay USDC to protocolTreasury and that pool's AccretionEngine
```

- beforeInitialize MUST reject pools using this hook unless the PoolId was registered by IndexLauncher.
- 5 bps LP fee is configured on the PoolKey (fee=500) and is independent of this hook.
- Emergency: guardian MAY disable fee take without unregistering the pool.

7.3 AccretionEngine.sol — BUILD

One clone per index. Holds swept USDC. harvest() is permissionless (or keeper-optional). v1.2 allocation: split engine USDC by launch weights (50/50 for AI2), buy each leg, then recognize worst-leg.

```
// harvest
usdcFor[i] = mulDiv(engineUsdc, launchWeightBps[i], 10_000) // 5000 / 5000
bought[i] = adapter.swapExactInput(USDC, asset[i], usdcFor[i], minOut[i], deadline)

// worst-leg recognition against current tracked (same as mint proportions after seed)
// compute the largest k such that k * tracked[i] / totalTrackedValue <= bought[i] for all i
// recognize recognized[i]; return leftover tokens + leftover USDC to the engine
index.depositAccretion(recognized) // MUST mint zero shares
```

If tNVDA rips and tMSFT does not, harvest still spends 50/50 USDC. The vault's live mix may drift. That is intended. Redeem is always pro-rata of whatever tracked then holds. There is no rebalance() in the MVP.

8. Invariants, tests, deployment

8.1 Hard invariants

- $\text{tracked}[i] \leq \text{raw balanceOf}(\text{vault}, i)$ after every state-changing call. syncLoss is the repair if raw falls (B20 seize).
- depositAccretion never increases totalSupply.
- seed / mint / redeem / depositAccretion are the only tracked writers besides syncLoss.
- Hook fee is USDC-only. LP fee never enters pendingHookUsdc.
- createIndex is onlyLauncher.
- In-kind redeem does not read the price feed.

8.2 Tests that MUST exist

- Unit: mint required-amount ceil, redeem floor, 10 bps share fee to treasury, seed fee-free.
- Unit: harvest 50/50 spend + worst-leg leftover stays in engine + supply flat.
- Unit: syncLoss never increases tracked; redeem after seize is solvent.
- Hook: buy takes from USDC in; sell takes from USDC out; exact-out reverts; unregistered pool cannot initialize.
- Invariant / fuzz: tracked $\leq$ raw; supply only moves on mint/redeem/seed; accretion does not mint.
- Integration (Sepolia or fork): createSeed $\rightarrow$ initializeMarket $\rightarrow$ swap $\rightarrow$ sweep $\rightarrow$ harvest $\rightarrow$ redeem.
- Match TypeScript golden vectors in engine.test.ts for the 50/50 fixture.

8.3 Deployment order

- Mocks + unit/invariant tests locally.
- On Sepolia: create tNVDA / tMSFT via B20 Factory; deploy mock feeds; seed constituent/USDC V4 pools.
- Deploy AccretiveIndex impl, AccretionEngine impl, adapter, zap, factory.
- Mine CREATE2 salt and deploy IndexFeeHook.
- Deploy IndexLauncher; authorize it on factory and hook; configure asset records and PoolKeys.
- Launch Index AI2 (two txs). Run DemoAccretion script.

8.4 Repository layout

```
src/
  core/      AccretiveIndex.sol  IndexFactory.sol
  launch/    IndexLauncher.sol
  periphery/ IndexZapRouter.sol  UniswapV4SwapAdapter.sol
  fees/      IndexFeeHook.sol
  accretion/ AccretionEngine.sol
  libs/      NavLib.sol  SqrtPriceLib.sol
  testnet/   MockPriceFeed.sol
test/
  unit/ fuzz/ invariant/ integration/
script/
  DeployTestB20s  DeployMockFeeds  SeedConstituentPools
  DeployProtocol  LaunchAI2  DemoAccretion
```

9. Frontend and acceptance

Recommended live stack: Next.js, TypeScript, viem, wagmi, Coinbase Wallet / WalletConnect. A Grok Build TypeScript simulator already walks the loop (launch, mint, trade, sweep, harvest, redeem, vault, B20 seize drill) and is the product-narrative reference. On-chain frontend MUST expose the same loop against Sepolia addresses.

Screen	Minimum
Launch	Two-tx createSeed + initializeMarket. Show 50/50 basket, 10 bps share fees, 5+5 fee split.
Vault	Tracked vs raw per constituent, totalSupply, navPerShare, cumulativeAccretedUsdWad. Ledger, not a step in the loop.
Mint	previewMint required basket / zap USDC path.

Screen	Minimum
Trade	Exact-in AI2 ↔ USDC on the canonical pool. Do not hide that LP fee and hook fee are separate.
Accretion	Pending hook, engine USDC, sweep, harvest. Fee-flow table as specified in §2.
Redeem	previewRedeem assetOut. In-kind first.
Disclaimer	Testnet only. tNVDA/tMSFT are synthetic.

9.1 Acceptance gate

- Addresses published for chain 84532.
- Scripted demo: launch → trade → sweep → harvest → redeem more basket per share than a pre-harvest redeem of the same shares.
- README: chain id, addresses, scripts, testnet disclaimer, reuse/source table from §0.3.
- No claim that test B20s are live Coinbase stocks.

9.2 Out of scope

- Mainnet, MAG7 UI, keepers, governance token, rebalance(), exact-output canonical swaps, multi-chain.
- US person / Reg S gating in contracts. Off-chain compliance is a later product decision, not this MVP.

Appendix A. Reference URLs

Reference	URL
Base B20 Native Token Standard	https://docs.base.org/upgrades/beryl/b20
Base Standard Library	https://github.com/base/base-std
Base B20 Factory docs	https://github.com/base/base-std/blob/main/docs/B20/Factory.md
Uniswap V4 deployments	https://docs.uniswap.org/contracts/v4/deployments
Uniswap V4 custom accounting	https://docs.uniswap.org/contracts/v4/guides/hooks/hook-returns
OpenZeppelin Uniswap Hooks	https://docs.openzeppelin.com/uniswap-hooks
Circle USDC addresses	https://developers.circle.com/stablecoins/usdc-contract-addresses
Vimen (MIT patterns)	https://github.com/vimenprotocol/vimen
Base Batches	https://www.basebatches.xyz

Appendix B. Worked 50/50 numeric sketch

Prices at seed: tNVDA = \$200, tMSFT = \$500. Launch spends 1,000 USDC backing 50/50.

Leg	USDC spent	Tokens bought (ideal, no slippage)
tNVDA	500	2.5 tNVDA
tMSFT	500	1.0 tMSFT

seed mints 1,000 Index AI2 fee-free (1 AI2 ≈ \$1 NAV). Later mint of G gross shares requires $\text{ceil}(\text{tracked}[i] * G / \text{supply})$ of each asset — current pile, not 50/50 USDC. Redeem of S shares (after 10 bps share fee) pays $\text{floor}(\text{tracked}[i] * \text{redeemShares} / \text{supply})$.

Example drift: tNVDA +50%, tMSFT unchanged. Vault NAV rises. A redeem of 1,000 AI2 is still 50/50 of the token pile, not 50/50 of USDC value. Harvest of 100 USDC still spends 50/50 USDC into the two legs, then worst-leg-recognizes against live tracked. Supply stays flat. That is the flywheel.

Appendix C. Agent implementation notes

- Pin foundry.toml versions. Re-read PoolManager + hook addresses from Uniswap’s current Base Sepolia table before deploy.

- Do not paste Uniswap or OpenZeppelin source into src/ as if it were ours.
- HookMiner / CREATE2 first. Wrong permission bits means a wasted deploy.
- Write the tracked $\leq$ raw invariant test before the happy-path demo script.
- Treat engine.ts vectors as failing CI if Solidity disagrees, not the other way around.
- If a behavior is missing from this document, open a spec question. Do not invent a third fee path.

Document control

v1.0 Developer Handoff (24 pp) established architecture and reuse matrix. v1.1 restored scope and set two-tx launch, 18-dec INDEX, mint-fee-to-treasury. v1.2 rebands the product as Omnibit Index Forge, locks Index AI2 at 50/50, separates LP fee from hook fee, and sets harvest spend to launch weights. This file is the binding spec.